AWS Terraform RAG Specification

Technical Specification

Natural-Language Network Architecture to AWS Terraform Translation

Control Rule

Primary output: AWS logical design and Terraform code only.

Core mapping:

- router -> EC2 instance with multiple network interfaces

- switch / lsw / vlan / lan segment -> subnet

- pc -> EC2 instance

- server -> EC2 instance unless the user explicitly requests a managed AWS service

- firewall -> explicit firewall mapping defined in this document

- routed device link -> explicit routed connectivity implemented through interfaces, subnets, route tables, security controls, and related AWS resources

Default access policy:

- Every EC2 that is not a router must receive a unique key pair and unique PEM file unless the user explicitly overrides this policy.

- Public SSH access must not be assumed by default.

- If public SSH is requested, SSH must be restricted by default to a variable such as admin_cidr, not 0.0.0.0/0, unless the user explicitly asks for insecure lab mode.

Connectivity behavior:

- Edges define connectivity and must not be invented.

- Missing edges block generation until clarified.

Addressing behavior:

- If addressing is missing, the system must ask for it.

- If the user explicitly says “do it by yourself”, automatic addressing is authorized.

- Automatic addressing must use VLSM.

Response order:

- Interpretation

- Mapping Table

- AWS Design Choice

- CIDR Plan

- SSH Access Plan

- Terraform Code

- Outputs

- Notes / Assumptions

1. Document Purpose

1.1 Purpose

This document defines the formal rules for a Retrieval-Augmented Generation system that converts a natural-language network architecture description into:

(1) an AWS logical design

and

(2) Terraform code.

1.2 Role of This Document

This document is the single source of truth for topology interpretation, AWS mapping, addressing behavior, SSH access behavior, Terraform generation behavior, clarification behavior, and forbidden assumptions.

1.3 Retrieval Objective

This document is written to support dense retrieval, BM25 retrieval, reranking, and chunk-based retrieval.

Therefore:

- rules are written explicitly

- mappings are repeated where needed

- each section is self-contained

- vague wording is avoided

- examples are included

1.4 Governing Principle

The system must follow the uploaded translation model unless the user explicitly overrides it.

The governing model for this specification is:

- router = EC2 instance with multiple interfaces

- switch / lsw / vlan / lan segment = subnet

- pc = EC2 instance

- server = EC2 instance unless explicitly overridden

- firewall = explicit firewall mapping defined in this document

- edges define connectivity and must not be invented

- missing mandatory information blocks generation

- automatic addressing is allowed only when the user explicitly authorizes it

- automatic addressing must use VLSM

2. Scope

2.1 In Scope

This specification covers:

- natural-language topology interpretation
- component extraction
- connectivity interpretation
- AWS resource mapping
- CIDR planning
- SSH key / PEM behavior
- Terraform generation
- output format
- ambiguity handling
- forbidden behavior

2.2 Out of Scope

This specification does not require:

- dynamic routing protocol emulation unless explicitly requested
- unsupported AWS services not justified by the topology
- unmanaged guessing of software roles
- exact reproduction of physical Layer 2 behavior beyond subnet abstraction
- generic substitution of logical components with unrelated AWS abstractions
- Ansible generation

2.3 Output Scope Rule

This version generates AWS logical design and Terraform output only.

The system must not generate Ansible unless the user explicitly asks for it.

3. Supported Input Entities

3.1 Supported Component Types

The system supports the following logical entities:

- router
- switch
- lsw
- vlan
- lan segment
- pc
- server
- firewall
- link
- edge
- router-to-router connection
- addressing information
- subnet information
- internet connectivity indication

3.2 Canonical Component Names

The system must normalize supported inputs into the following canonical logical types:

- router
- switch
- pc

- server

- firewall

3.3 Normalization Rules

The following normalizations are allowed when unambiguous:

- lsw -> switch

- vlan -> switch-equivalent Layer 2 segment

- lan segment -> switch-equivalent Layer 2 segment

- desktop -> pc

- workstation -> pc

3.4 Unsupported Ambiguity Rule

If an entity cannot be mapped confidently, the system must state the ambiguity and stop if that ambiguity materially affects correctness.

3.5 No Device Invention Rule

The system must not invent devices that are not justified by the topology.

4. Topology Interpretation Rules

4.1 Graph Interpretation Rule

The system must interpret the topology as a graph of nodes, links, segment memberships, routed boundaries, firewall placement, and optional addressing information.

4.2 Node Rule

Each described component becomes one logical node unless the user explicitly indicates that two labels refer to the same object.

4.3 Edge Rule

Each stated connection becomes one logical edge.

4.4 No Edge Invention Rule

The system must not invent links that are not justified by the input.

4.5 Segment Membership Rule

If a host is connected to a switch, that host belongs to the subnet derived from that switch.

4.6 Router Boundary Rule

If a switch is connected to a router, that subnet is attached to the router through one interface.

4.7 Layer 2 Interpretation Rule

A switch, LSW, VLAN, or LAN segment must be interpreted as a subnet-level grouping inside AWS.

4.8 Router Interpretation Rule

A router must be interpreted as a routed EC2 appliance with one AWS network interface per required connection.

A router must not be interpreted as a VPC by default.

4.9 Firewall Interpretation Rule

A firewall must be interpreted only according to the explicit firewall mapping defined in this specification.

4.10 Interpretation Shortcut Rules

The system must not collapse a router into a generic AWS abstraction such as a VPC if the direct mapping requires an EC2 instance with multiple interfaces.

The system must preserve the logical device roles given in the user topology.

5. Mandatory Information Required Before Generation

5.1 Minimum Required Input

The system must have enough information to determine:

- the components present
- the type of each component
- the edges between components
- router interface counts when needed
- addressing information, or explicit authorization for automatic addressing

5.2 Blocking Rule

If mandatory information is missing, the system must not generate Terraform.

5.3 Missing Edge Rule

If an edge is missing, unclear, or incomplete, the system must ask a targeted clarification question and wait.

5.4 Missing Addressing Rule

If addressing is missing, the system must ask for addressing information unless the user explicitly says “do it by yourself” or an equivalent authorization.

5.5 Explicit Auto-Addressing Authorization Rule

Automatic addressing is allowed only when the user explicitly authorizes it.

5.6 No Proceed-on-Assumption Rule

The system must not proceed with material topology assumptions that affect correctness.

6. Missing-Information Handling Policy

6.1 General Policy

When information is incomplete, the system must ask targeted clarification questions.

6.2 Targeted Clarification Rule

Clarification questions must be specific.

Valid examples:

- “What is PC1 connected to?”
- “Is firewall FW1 connected between R1 and SW1?”
- “What is the interface count of router R1?”
- “Please provide CIDRs, or say ‘do it by yourself’ if you want automatic VLSM.”

Invalid examples:

- “Please clarify the architecture.”
- “Can you give more details?”

6.3 Generation Blocking Rule

The system must not generate Terraform until the topology is valid and complete.

6.4 No Hidden Defaults Rule

The system must not silently assume missing edges, missing firewall placement, missing router interfaces, or missing addressing.

7. Edge and Connectivity Rules

7.1 Edge Meaning

An edge defines intended connectivity.

7.2 Host-to-Switch Rule

If a PC or server is connected to a switch, that host belongs to the subnet derived from that switch.

7.3 Switch-to-Router Rule

If a switch is connected to a router, that subnet belongs to one router interface domain.

7.4 Router-to-Router Rule

If two routers are connected, the system must interpret that as required routed connectivity between router interfaces.

7.5 Router-Link Translation Rule

Router-to-router links must be implemented through explicit subnets, interfaces, and routing logic according to the AWS design generated from the specification.

7.6 Multi-Segment Router Rule

If one router connects to multiple switches or LAN segments, those must become multiple subnets attached to that same router.

7.7 No Direct Collapsing Rule

The system must not collapse distinct routers into a single AWS abstraction unless the user explicitly indicates they are not separate routing domains.

8. Addressing Rules

8.1 User-Provided Addressing Rule

If the user provides valid addressing, the system must use it.

8.2 Missing Addressing Rule

If the user does not provide addressing, the system must ask for it unless the user explicitly authorizes automatic addressing.

8.3 Automatic Addressing Authorization Rule

The system may generate addressing automatically only when the user explicitly says “do it by yourself” or an equivalent instruction.

8.4 VLSM Requirement Rule

When automatic addressing is authorized, the system must use VLSM.

8.5 Non-Overlap Rule

Distinct subnets must not overlap.

8.6 Validity Rule

Subnet CIDRs must be valid and consistent with the topology.

8.7 Switch Subnet Rule

Each switch-derived segment must receive one subnet.

8.8 Point-to-Point Rule

Point-to-point links between routed devices should receive dedicated small subnets, typically /30 unless the design explicitly requires another size.

8.9 Gateway Rule

LAN subnets should reserve a gateway IP according to the addressing algorithm defined in this document.

Point-to-point links do not require an extra gateway reservation beyond the endpoint IPs.

9. Automatic Addressing and VLSM Policy

9.1 Activation Rule

Automatic addressing is activated only when the user explicitly authorizes it.

9.2 Mandatory Method Rule

Automatic addressing must use VLSM.

9.3 Base CIDR Rule

If the user provides a base CIDR, the system must use it.

If the user authorizes automatic addressing but does not provide a base CIDR, the default base CIDR is 10.0.0.0/16 unless overridden by the user.

9.4 Allocation Rule

Subnet allocation must be deterministic and must allocate larger requirements first.

9.5 LAN Sizing Rule

Each switch-derived LAN must receive a subnet sized for:

- all end devices in the segment

- all infrastructure devices attached to the segment

- the required gateway reservation

9.6 Point-to-Point Sizing Rule

Each routed point-to-point link must receive a subnet sized for the two endpoints.

No extra gateway reservation is required for point-to-point links.

9.7 Assignment Rule

The system must produce explicit IP assignments for the generated design.

10. AWS Translation Rules for Each Component Type

10.1 Fixed Mapping Rule

The system must apply the following mappings unless the user explicitly overrides them:

- router -> EC2 instance with multiple network interfaces

- switch / lsw / vlan / lan segment -> subnet

- pc -> EC2 instance

- server -> EC2 instance unless explicitly overridden

- firewall -> explicit firewall mapping defined in this specification

- routed device link -> explicit routed connectivity implemented through interfaces, subnets, route tables, and related AWS resources

10.2 Server Managed-Service Override Rule

If the user explicitly says a server should be managed, the system may replace it with a managed AWS service.

Otherwise, the default remains EC2.

10.3 Native AWS Preference Rule

The system should prefer native AWS constructs only when those constructs do not violate the direct mapping rules defined in this specification.

10.4 No Direct-Mapping Override Rule

The system must not replace a direct mapping with a generic AWS abstraction unless explicitly required by the document or the user.

11. Router Translation Rules

11.1 Router Mapping Rule

Each router must become one EC2 instance.

11.2 Multi-Interface Rule

Each router must have one AWS network interface per required connection implied by the validated topology.

11.3 Interface Count Rule

If the user explicitly provides the router interface count, the system must validate that it is sufficient for the router's edges.

11.4 No Router-to-VPC Rule

A router must not be translated into a VPC by default.

11.5 Placement Rule

Router interfaces must be attached to the correct subnets according to the topology.

11.6 Routing Rule

If the router is responsible for forwarding traffic, the generated design must include the route logic required by the translated AWS architecture.

11.7 Router EC2 Implementation Rule

For every router EC2, the design must explicitly include:

- source/destination check disabled

- IP forwarding enabled

- the network interfaces required by the topology

- operating-system routing configuration consistent with the generated path design

11.8 Router Configuration Delivery Rule

If router OS routing commands or sysctl configuration are needed, Terraform must deliver them using user_data or cloud-init unless the user requests another mechanism.

12. Switch-to-Subnet Rules

12.1 Switch Mapping Rule

Each switch, LSW, VLAN, or LAN segment must become one subnet.

12.2 Membership Rule

Each host connected to that switch must be placed in that subnet.

12.3 Distinct Segment Rule

Distinct switch labels should be treated as distinct subnets unless the user explicitly states they represent the same segment.

12.4 No EC2 Switch Rule

A switch must not be translated into an EC2 instance.

13. PC and Server Translation Rules

13.1 PC Mapping Rule

Each PC must become one EC2 instance.

13.2 Server Mapping Rule

Each server must become one EC2 instance unless explicitly overridden.

13.3 Placement Rule

A PC or server must be placed in the subnet derived from the switch or segment it belongs to.

13.4 Key Pair Rule

Every EC2 that is not a router must receive a unique key pair and unique PEM file unless the user explicitly overrides this rule.

13.5 Public Access Rule

Public exposure must not be assumed by default unless this document or the user explicitly requires it.

13.6 Instance Defaults Rule

If no instance type is provided, the default must be t3.micro.

If no AMI is provided, the system must use a clearly marked placeholder variable.

14. Firewall Translation Rules

14.1 Explicit Mapping Rule

A firewall must be translated only according to the mapping explicitly defined in this specification or by the user.

14.2 Supported Firewall Modes

The firewall translation must use one of these explicit modes:

- sg

- aws_network_firewall

- appliance

14.3 Default Firewall Mode Rule

If the user mentions a firewall but does not specify its implementation mode, the system must ask which firewall mode is desired.

The system must not silently choose one.

14.4 Firewall Mode: sg

If firewall mode is sg:

- use Security Groups and, if needed, NACLs

- preserve the logical filtering intent as closely as possible

- do not claim deep packet inspection

14.5 Firewall Mode: aws_network_firewall

If firewall mode is aws_network_firewall:

- use AWS Network Firewall resources

- preserve the stated traffic path as closely as possible

- include the route logic needed to steer traffic through the firewall path

14.6 Firewall Mode: appliance

If firewall mode is appliance:

- use an EC2-based firewall appliance

- attach the required interfaces

- disable source/destination check if forwarding is required

- include the route logic needed to preserve the topology

14.7 Connectivity Preservation Rule

If a firewall appears between two components in the topology, the translated AWS design must preserve that logical placement as closely as possible.

15. Routing and Connectivity Behavior in AWS

15.1 Explicit Route Rule

Route tables and route logic must be explicit in Terraform.

The system must not leave routing implicit in the explanation.

15.2 Internet Access Rule

If public SSH access is required, the design must include the necessary public-access resources and route logic.

15.3 Multi-Subnet Routing Rule

If the design has multiple subnets and routed boundaries, the routing must be configured consistently with the intended topology.

15.4 Connectivity Preservation Rule

The translated AWS routing should preserve the logical connectivity implied by the real topology as closely as possible.

16. Key Pair / PEM Generation Policy

16.1 Unique Key Rule

Every EC2 that is not a router must receive its own unique SSH key pair by default.

16.2 Terraform Key Generation Rule

For each applicable EC2 instance, Terraform must create:

- one tls_private_key
- one aws_key_pair
- one local_file for the PEM

16.3 Naming Rule

Key and PEM resources should be named clearly using the instance name.

16.4 Attachment Rule

Each EC2 instance must be attached to its matching key pair.

16.5 Output Rule

The final output must state which PEM file belongs to which EC2 instance.

17. SSH Access Behavior

17.1 Explicit SSH Rule

SSH exposure must follow the user's request or the explicit policy in the document.

17.2 No Forced Public Exposure Rule

The system must not assume public SSH access by default unless explicitly required.

17.3 Secure Default SSH Rule

If public SSH access is requested and the user does not specify a source CIDR, the system must use a Terraform variable named admin_cidr and restrict SSH to that CIDR by default.

17.4 Insecure Lab Mode Rule

SSH from 0.0.0.0/0 may be used only if the user explicitly asks for insecure lab mode.

17.5 Public SSH Implementation Rule

If public SSH access is configured, the design must include:

- public subnet behavior or public IP association
- internet gateway
- route table with 0.0.0.0/0 to the internet gateway
- security rules allowing TCP/22 from the selected source CIDR

17.6 SSH Command Output Rule

If public SSH access is configured, the system should print example SSH commands such as:

- ssh -i pc1.pem ec2-user@<pc1_public_ip>

17.7 Username Assumption Rule

If the AMI family is not specified, the system should mention the likely SSH username assumption.

17.8 Private-Only Rule

If the user explicitly asks for private-only instances, the public SSH defaults must not be applied.

18. Terraform Generation Requirements

18.1 Required Generation Content

When generating output, the system must produce:

- AWS logical interpretation
- Terraform code
- addressing summary
- key pair / PEM summary

18.2 Required Terraform Content

Terraform must include the required AWS resources, route logic, EC2 resources, subnet resources, key resources, variables where useful, and outputs.

18.3 Required Key Resources

Terraform must use tls_private_key, aws_key_pair, and local_file for each required EC2 key pair and PEM generation.

18.4 AMI Rule

Avoid hardcoding AMI IDs unless they are passed through variables or explicitly provided by the user.

18.5 Tagging Rule

All resources should be tagged clearly with topology names such as router, switch, PC, server, and firewall names.

18.6 Minimal Edit Rule

Terraform should be runnable with minimal edits.

19. Output Structure Requirements

19.1 Mandatory Response Order

Whenever the user gives a topology, the system must respond in this order:

Interpretation, Mapping Table, AWS Design Choice, CIDR Plan, SSH Access Plan, Terraform Code, Outputs, Notes / Assumptions.

19.2 Explanation Before Code

Before giving Terraform code, the system must provide:

- a short interpretation of the topology
- a mapping table from devices to AWS resources
- the AWS design chosen
- the CIDR plan
- the SSH access plan

19.3 Assumption Disclosure Rule

If assumptions were made, they must be stated explicitly in the Notes / Assumptions section.

20. Validation Rules

20.1 Structural Validation Rule

The system must validate that the topology can be translated consistently according to the mapping model.

20.2 Mapping Validation Rule

The system must ensure that:

- routers become EC2 instances with the required interfaces

- switches become subnets

- PCs become EC2 instances

- servers become EC2 instances unless explicitly overridden

- firewalls follow the explicit firewall mapping defined in this document

20.3 Address Validation Rule

The system must ensure:

- subnet CIDRs are valid

- subnet CIDRs do not overlap

- addressing is consistent with topology needs

20.4 Routing Validation Rule

The system must ensure routing and connectivity are consistent with the translated topology.

20.5 Assumption Validation Rule

If assumptions are used, they must be clearly stated, reasonable, and consistent with the governing rules.

20.6 Blocking Validation Rule

Missing mandatory details must block generation unless the missing detail is explicitly covered by an allowed rule such as user-authorized automatic addressing.

21. Forbidden Assumptions and Forbidden Behaviors

21.1 No Unjustified Devices

The system must not invent devices that are not justified.

21.2 No Router-to-VPC Substitution

The system must not replace a router with a VPC unless the user explicitly overrides the mapping.

21.3 No Firewall Auto-Substitution

The system must not replace a firewall with a Security Group, AWS Network Firewall, or EC2 appliance unless the selected firewall mode explicitly allows it.

21.4 No Edge Invention

The system must not invent missing edges.

21.5 No Hidden Addressing

The system must not invent addressing unless the user explicitly authorizes automatic addressing.

21.6 No Response Before Validation

The system must not generate Terraform before validation succeeds.

21.7 No Hidden Assumptions

If any assumption is used, it must be explicitly stated.

22. Error-Handling and Clarification Examples

22.1 Example: Missing Edges

Input: pc1, switch1, router1.

Required behavior:

- ask what pc1 is connected to
- ask what switch1 is connected to
- ask what router1 is connected to if not stated
- do not generate code yet

22.2 Example: Missing Addressing

Input: pc1--switch1, switch1-router1.

Required behavior:

- ask for addressing
- or instruct the user to say “do it by yourself” if automatic VLSM is desired

22.3 Example: Auto Addressing Authorization

Input: pc1--switch1, switch1-router1, do it by yourself.

Required behavior:

- activate automatic addressing
- use VLSM
- continue if all other mandatory topology information is present

22.4 Example: Ambiguous Firewall Placement

Input: pc1--switch1, router1, firewall1.

Required behavior:

- ask where firewall1 is placed

- do not invent firewall placement

22.5 Example: Ambiguous Firewall Mode

Input: pc1--fw1--r1.

Required behavior:

- ask whether fw1 should be implemented as sg, aws_network_firewall, or appliance

- do not silently choose one

23. End-to-End Examples

23.1 Example Topology 1: Same Router, One LAN

Input: pc1--sw1 pc2--sw1 s1--sw1 fw1--sw1 r1--fw1.

Interpretation:

- pc1, pc2, and s1 are EC2 instances

- sw1 is one subnet

- r1 is one EC2 router with multiple interfaces

- fw1 follows the selected firewall mode

- the firewall is logically between r1 and sw1

23.2 Example Topology 2: Router with Multiple LANs

Input: pc1--sw1 pc2--sw2 sw1-r1 sw2-r1.

Interpretation:

- r1 is one EC2 router

- sw1 and sw2 are distinct subnets

- pc1 and pc2 are EC2 instances

- r1 connects to both subnets through multiple interfaces

23.3 Example Topology 3: Two Routers with Direct Link

Input: pc1--sw1 sw1-r1 pc2--sw2 sw2-r2 r1-r2.

Interpretation:

- r1 and r2 are distinct EC2 router instances

- sw1 and sw2 are distinct subnets

- r1-r2 is a routed point-to-point segment

- explicit routing logic is required

23.4 Example Topology 4: Manual Addressing

Input:

pc1--sw1

s1--sw1

fw1--sw1

r1--fw1

base cidr 10.0.0.0/16

sw1 = 10.0.1.0/29

Interpretation:

- sw1 receives the provided subnet

- r1 is an EC2 router

- addressing must use the user-provided values

- no automatic VLSM is needed

24. Glossary

24.1 Router

A routed network device.

In this specification, a router is translated to an EC2 instance with multiple interfaces.

24.2 Switch

A Layer 2 segment boundary.

In this specification, a switch, LSW, VLAN, or LAN segment is translated to a subnet.

24.3 PC

An end-user host.

In this specification, a PC is translated to an EC2 instance.

24.4 Server

A service host.

In this specification, a server is translated to an EC2 instance unless the user explicitly requests otherwise.

24.5 Firewall

A security-control device.

In this specification, a firewall is translated only according to the explicit firewall mode selected.

24.6 Edge

A stated connectivity relation between topology entities.

24.7 PEM File

A private key file used for SSH access to EC2 instances.

24.8 Terraform

The infrastructure-as-code output generated by the system to build the AWS design.

Appendix A. High-Priority Retrieval Rules

1. router = EC2 instance with multiple interfaces.

2. router must not be translated as a VPC by default.

3. switch / lsw / vlan / lan segment = subnet.

4. pc = EC2.

5. server = EC2 unless explicitly overridden.

6. firewall must use an explicit selected mode.

7. firewall must not silently default to Security Group.

8. edges define connectivity and must not be invented.

9. missing edges block generation.

10. missing addressing requires clarification unless the user explicitly authorizes automatic addressing.

11. automatic addressing must use VLSM.

12. every EC2 that is not a router must have a unique key pair and PEM by default.

13. generate Terraform only unless the user explicitly asks for more.

14. do not replace direct mappings with generic AWS abstractions.

15. response order must include Terraform Code and must not include Ansible unless requested.